

DATA MODELING USING THE ENTITY- RELATIONSHIP(ER) MODEL



SAMPLE DB APPLICATION

- The company is organized into DEPARTMENTS. Each department has a name, number and an employee who *manages* the department. We keep track of the start date of the department manager. A department may have several locations.
- Each department *controls* a number of PROJECTs. Each project has a unique name, unique number and is located at a single location.

- We store each EMPLOYEE's social security number, address, salary, sex, and birthdate.
 - Each employee *works for* one department but may *work on* several projects.
 - We keep track of the number of hours per week that an employee currently works on each project.
 - We also keep track of the *direct supervisor* of each employee.
- Each employee may *have* a number of DEPENDENTS.
 - For each dependent, we keep track of their name, sex, birthdate, and relationship to the employee.

1. Consider the schema for Employee Database:

EMPLOYEE(SSN, FName, LName, Address, Sex, Salary, SuperSSN, DNo)

DEPARTMENT(DNo, DName, MgrSSN, MgrStartDate)

DLOCATION(DNo, DLoc)

PROJECT(PNo, PName, PLocation, DNo)

WORKS_ON(SSN, PNo, Hours)

DEPENDENT(ESSN, Name, Sex, Bdate, Relationship)

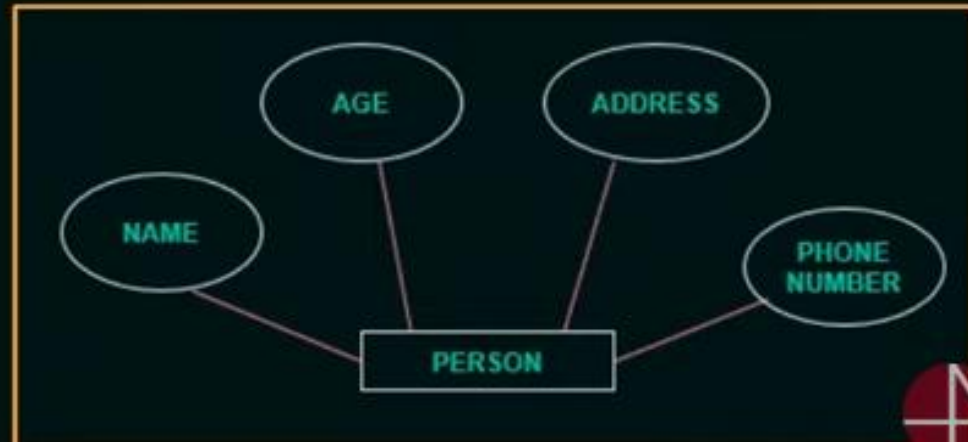
❖ Terminologies

★ Entity:

- A “thing” in the real world with an **independent existence**.
- May be an object with physical existence (**ex:** house, person) or with a conceptual existence(**ex:** course, job).

★ Attributes:

- Properties that **describe the entities**.



❖ Terminologies

Composite Attributes

- ★ Can be divided into further parts.
- ★ Ex: Name → First Name, Middle Name, Last Name

Simple Attributes

- ★ Cannot be divided further.
- ★ Ex: Weight → cannot be further divided.

❖ Terminologies

Single-Valued Attributes

- ★ Have a **single value** for a particular entity.
- ★ **Ex:** Age → single-valued attribute of a person.

Multivalued Attributes

- ★ Can have **set of values** for a particular entity.
- ★ **Ex:** College degree, languages known → multivalued attributes of a person.



❖ Terminologies

Derived Attributes

- ★ Can be derived from other attributes.
- ★ **Ex:** Age → can be derived from date of birth.

Stored Attributes

- ★ From which the value of other attributes are derived.
- ★ **Ex:** BirthDate of a person

❖ Terminologies

★ Complex Attributes:

- Has multivalued & composite components in it.
- **Multivalued** attributes → represented within '{ }'.
- **Composite** attributes → represented within '()'.
- **Ex:** {CollegeDegrees(College, Year, Degree, Field)}

★ Null Values:

- Null is something which is **not applicable** or **unknown**.

STUDENT_ID	NAME	AGE	PHONE_NO
1	HARRY	20	123546897
2	JEREMY	22	NULL
3	JOSHUA	NULL	2525468912

❖ Terminologies

★ Entity Type:

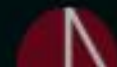
- A collection of entities that have the same attributes.
- Ex: STUDENT

STUDENT

STUDENT_ID	NAME	AGE
1	HARRY	20
2	JEREMY	22
3	JOSHUA	18

★ Entity Set:

- Collection of entities of a particular entity type at a point in time.



❖ Terminologies

★ Key Attribute:

- That attribute that is capable of **identifying** each entity **uniquely**.
- **Ex:** Roll number of a student

STUDENT

STUDENT_ID	NAME	AGE
1	PRIYANKA	20
2	JEREMY	21
3	PRIYANKA	20

★ Value Set of Attributes:

- The set of values that can be assigned to an attribute.

COMPOSITE KEY EXAMPLE

- Consider a table with three attributes *customer ID, product ID, product quantity*
- Customer ID needs to be entered for each time the customer purchases an order hence customer ID appears more than once in the customer ID table hence it **cannot be served as the primary key i.e it failed to uniquely identify a record**
- Example customer ID 66 has placed two orders hence customer ID appeared 66 two times in the customer ID column

Customer_ID	Product_ID	Order_Quantity
66	9023	10
67	9023	15
68	9031	20
69	9031	18
66	9111	50

- Product ID and product quantity cannot be declared as the primary key **because of more than one customer purchase same product and the same quantity**
- In this situation all three attributes fail to serve as a primary key. Hence **combination of these attribute can be used as a primary key**
- For example **[customer ID, product ID] can be used as the primary key** table customers this combination helps to uniquely access records of customer.

VALUE SETS(DOMAINS) OF ATTRIBUTES

- Each simple attribute of an entity types is associated with a **value set or domain of values**, which specifies a set of values that may be assigned to that attribute.
- If the range of ages allowed for employees between 18 to 70, we can specify the value set of the Age attribute of EMPLOYEE to be the set of integer numbers between 18 to 70.
- Similarly, we can specify the value set for the Name attribute as being the set of strings of alphabetic characters separated by blank characters and so on.
- **Value sets are not specifically represented in the ER Diagram.**
- They are similar to data types.

INITIAL CONCEPTUAL DESIGN OF THE COMPANY DATABASE

- We can identify four initial entity types in the COMPANY database:
 - DEPARTMENT
 - PROJECT
 - EMPLOYEE
 - DEPENDENT



Entity



Relationship



Attribute



Weak Entity



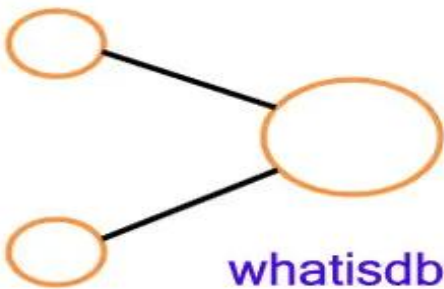
Weak Entity
Relationship



Multivalued
Attribute



Key Attribute



Composite
Attribute

INITIAL DESIGN OF ENTITY TYPES: EMPLOYEE, DEPARTMENT, PROJECT, DEPENDENT

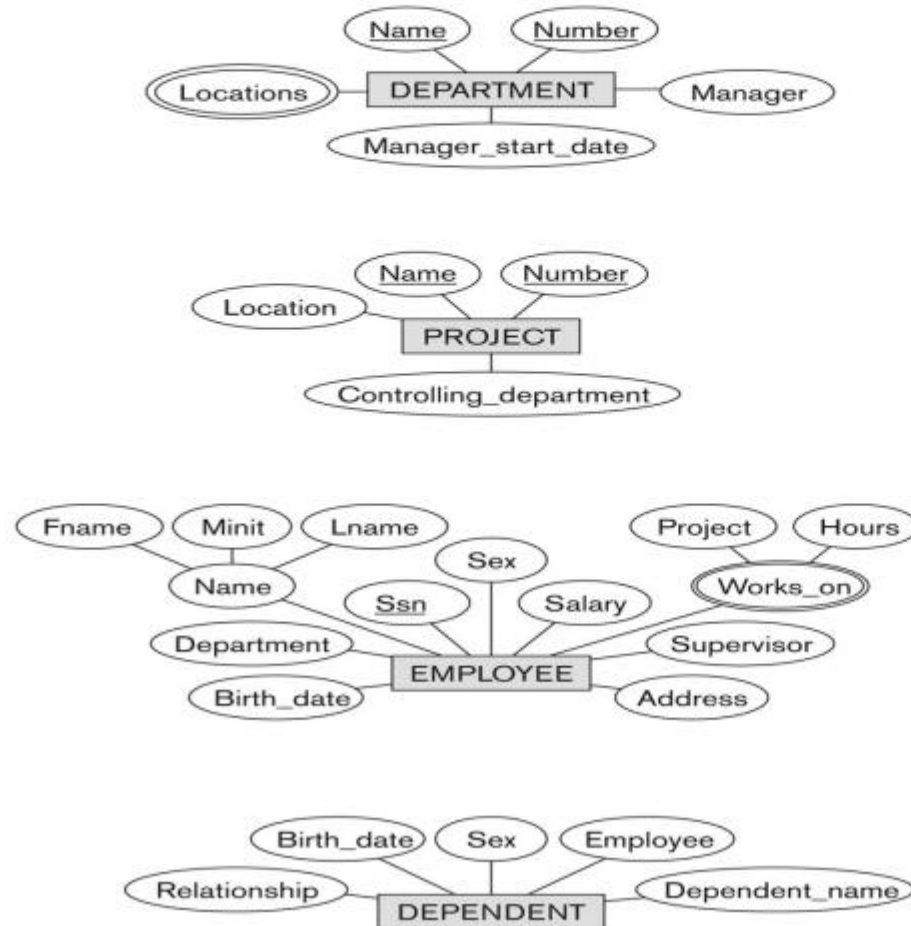


Figure 3.8
Preliminary design of entity
types for the COMPANY
database. Some of the
shown attributes will be
refined into relationships.

REFINING THE INITIAL DESIGN BY INTRODUCING RELATIONSHIPS

- The initial design is typically not complete
- ER model has three main concepts:
 - **Entities** (and their entity types and entity sets)
 - **Attributes** (simple, composite, multivalued)
 - **Relationships** (and their relationship types and relationship sets)

- **Relationships:** A **relationship** relates two or more distinct entities with a specific meaning.
 - For example, EMPLOYEE John Smith *works on* the ProductX PROJECT, or EMPLOYEE Franklin Wong *manages* the Research DEPARTMENT.
- **Relationship Types:** Relationships of the same type are grouped or typed into a **relationship type**.
 - For example, the WORKS_ON relationship type in which EMPLOYEEs and PROJECTs participate, or the MANAGES relationship type in which EMPLOYEEs and DEPARTMENTs participate.

▪ **Relationship Instances:**

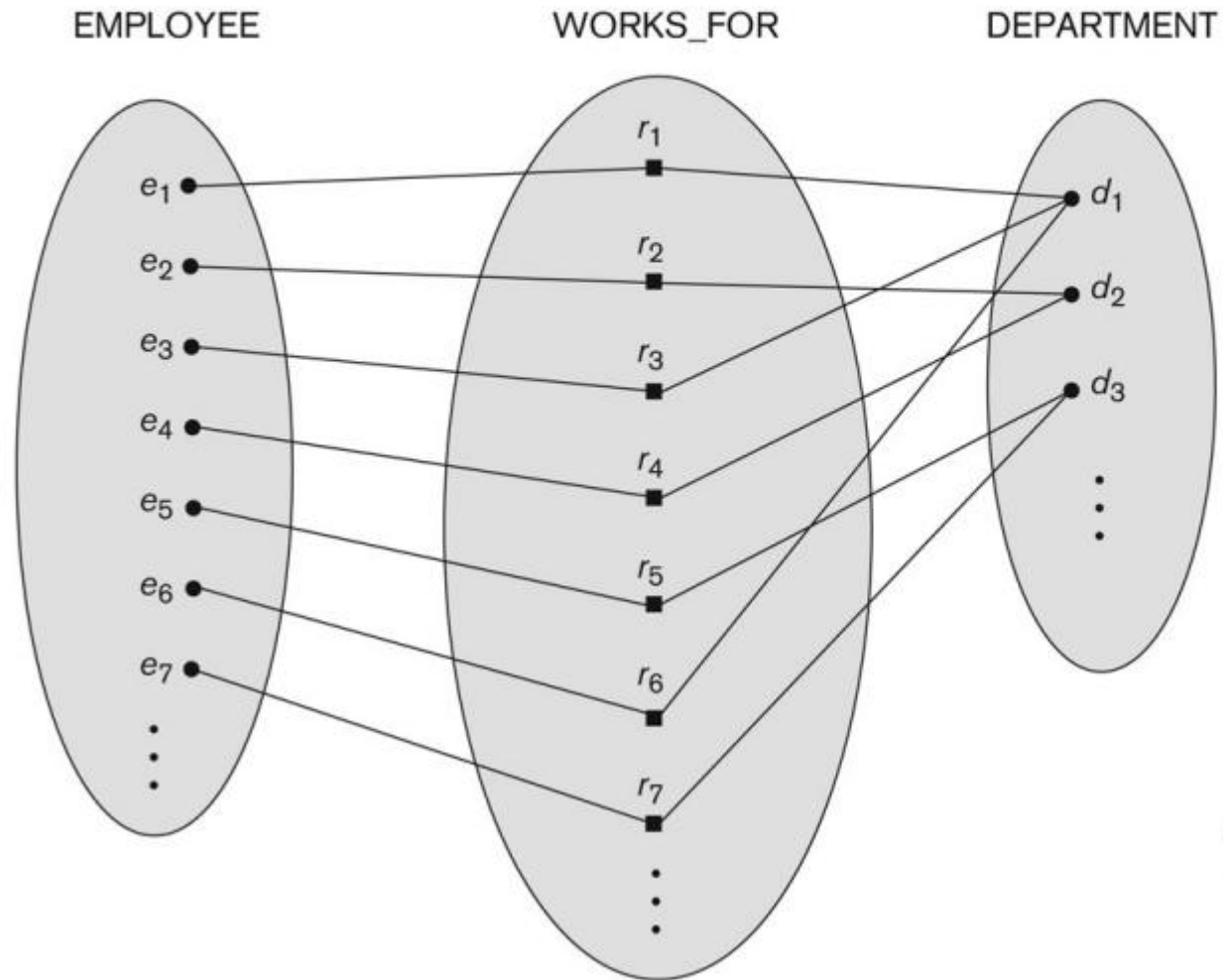


Figure 3.9

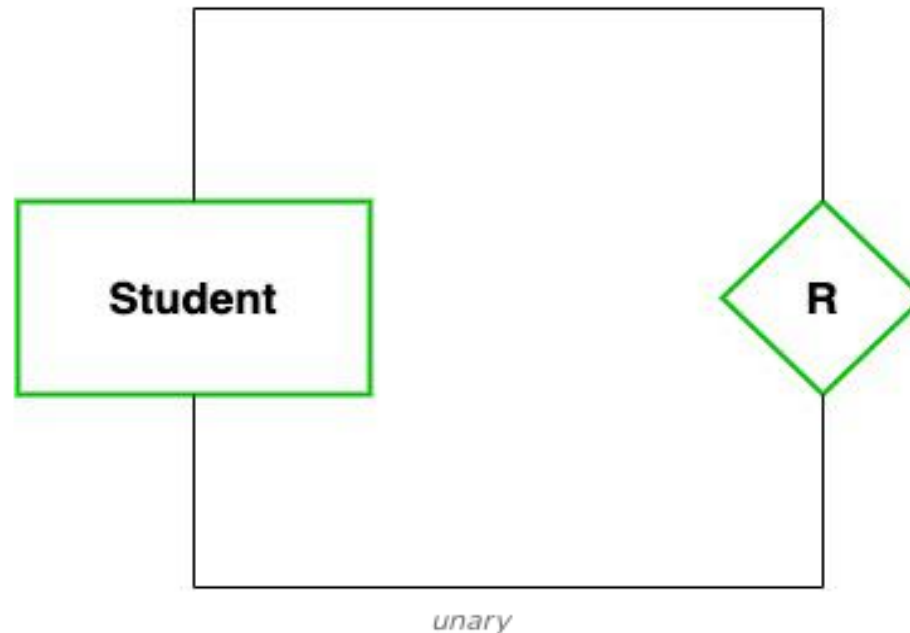
Some instances in the WORKS_FOR relationship set, which represents a relationship type WORKS_FOR between EMPLOYEE and DEPARTMENT.

- In ER diagram, Relationship types are displayed as diamond-shaped boxes, which are connected by straight lines to the rectangular boxes representing the participating entity types.
- In the above figure WORKS_FOR is **relationship type** and r1, r2,r3,.....r7 are **relationship instances**.
- **Relationship Degree:** The degree of the relationship is the number of participating entity types.
- A relationship type with degree 1 is termed unary
- A relationship type with degree 2 is termed binary.
- A relationship type with degree 3 is termed ternary.

Unary (Degree 1)

In this type of relationship, both the associating entity types are the same. So, we can say that unary relationships exist when both entity types are the same and we call them the degree of relationship is 1. In other words, in a relation only one entity set is participating then such type of relationship is known as a unary relationship.

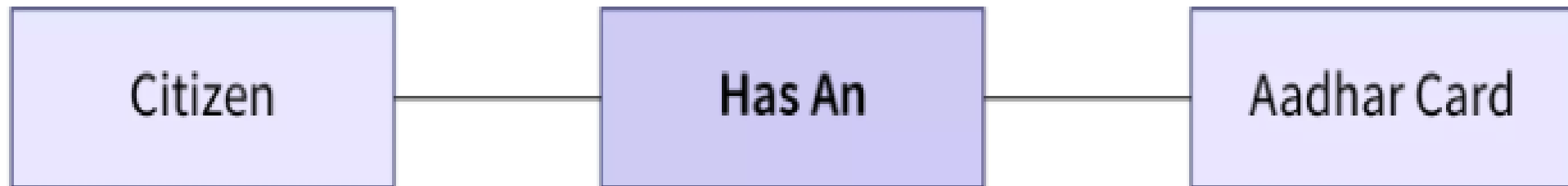
Example: In a particular class, we have many students, there are monitors too. So, here class monitors are also students. Thus, we can say that only students are participating here. So the degree of such type of relationship is 1.



Binary Relationship

You must have rightly guessed that in a binary relationship, there are two entities involved. There are two entities that are participating in the relationship. The degree of relationship is 2. This is the most common type of relationship.

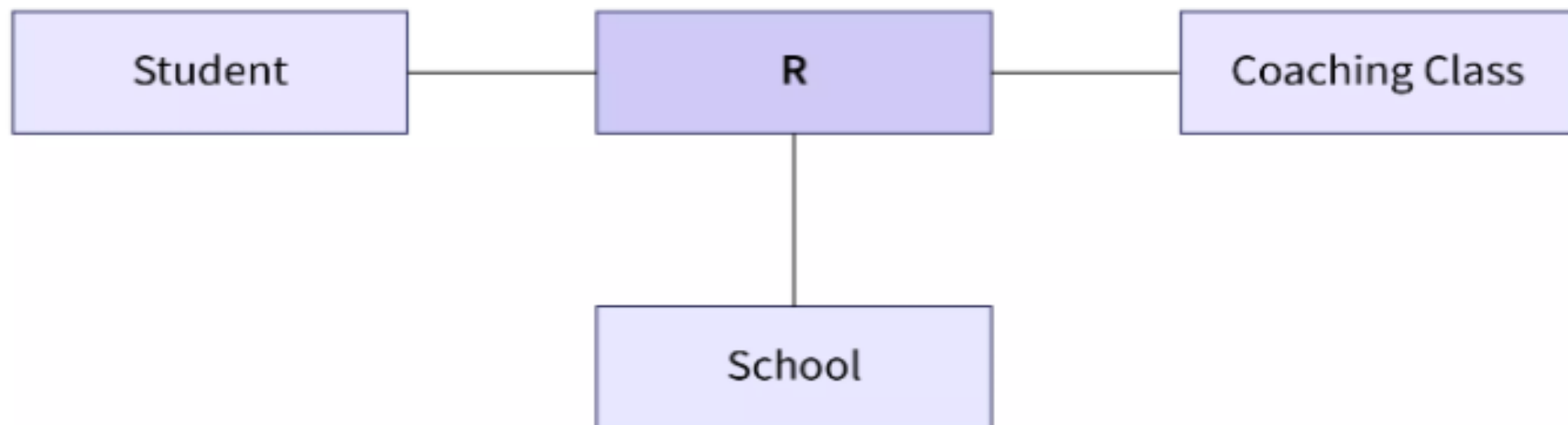
Example- Each Indian citizen has their own Aadhar Card so we can say that citizen and Aadhar Card are two entities involved.



Ternary Relationship

In a **ternary** relationship, there are three entities involved. There are three entities that are participating in the relationship created. The degree of the relationship is 3. As in this relationship, the number of entities increases, it is a bit difficult to convert it to a **relational** table.

Example- A student studies in a school, so student and school are two entities. But, the same student also takes some coaching classes, so he has a relation with coaching as well, so the coaching class is also an entity. Here as there are **3** entities, so it is a **ternary** relationship.



- **Role Names :**

- Each Entity type that participates in a relationship plays a particular role in the relationship.
- The **role name** signifies the role that a participating entity from the entity type plays in each relationship instance.
- Example: Suppose we have two Entity types EMPLOYEE and DEPARTMENT and the relationship is WORKS_FOR. Then EMPLOYEE plays the role of an worker and DEPARTMENT plays the role of an employer.

- **Recursive Relationships:**

- Same Entity types participates more than once in a relationship type in different roles, in such cases the role name becomes essential for distinguishing the meaning of each participation. Such relationship types are called recursive relationships..
- Also called as **self-referencing** relationships.
- Example:
 - Relationship: SUPERVISION
 - Participating Entity type: EMPLOYEE
 - EMPLOYEE Entity type participates twice in the relationship SUPERVISION once in the role of a supervisor and once in the role of a supervisee.

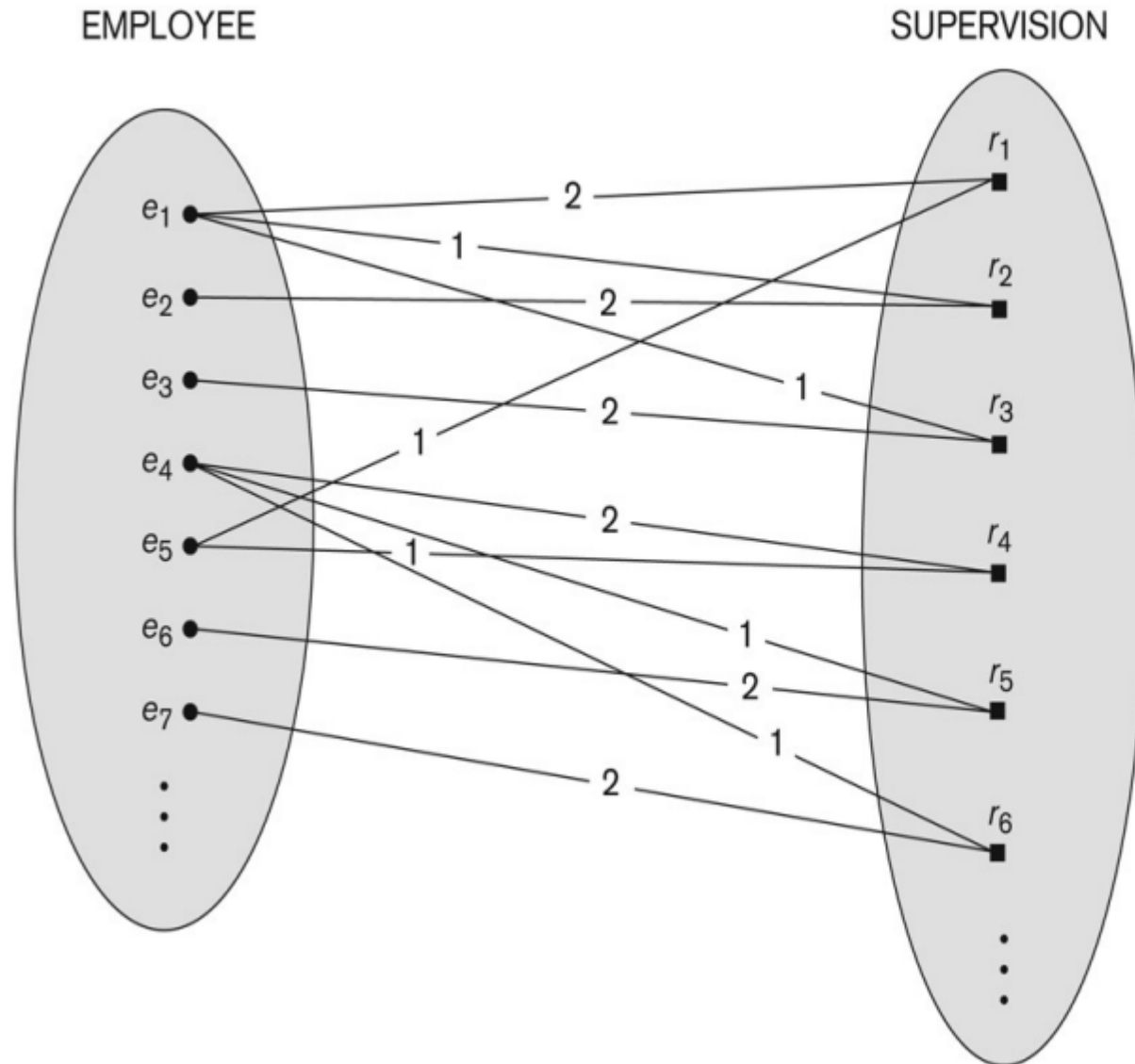


Figure 3.11

A recursive relationship SUPERVISION between EMPLOYEE in the *supervisor* role (1) and EMPLOYEE in the *subordinate* role (2).

❖ Relationships

★ Relationship Constraints:

1. Cardinality Ratio

- Maximum number of relationship instances that an entity can participate in.
- Possible cardinality ratios for binary relationship $\rightarrow$ 1:1, 1:N, N:1, M:N.



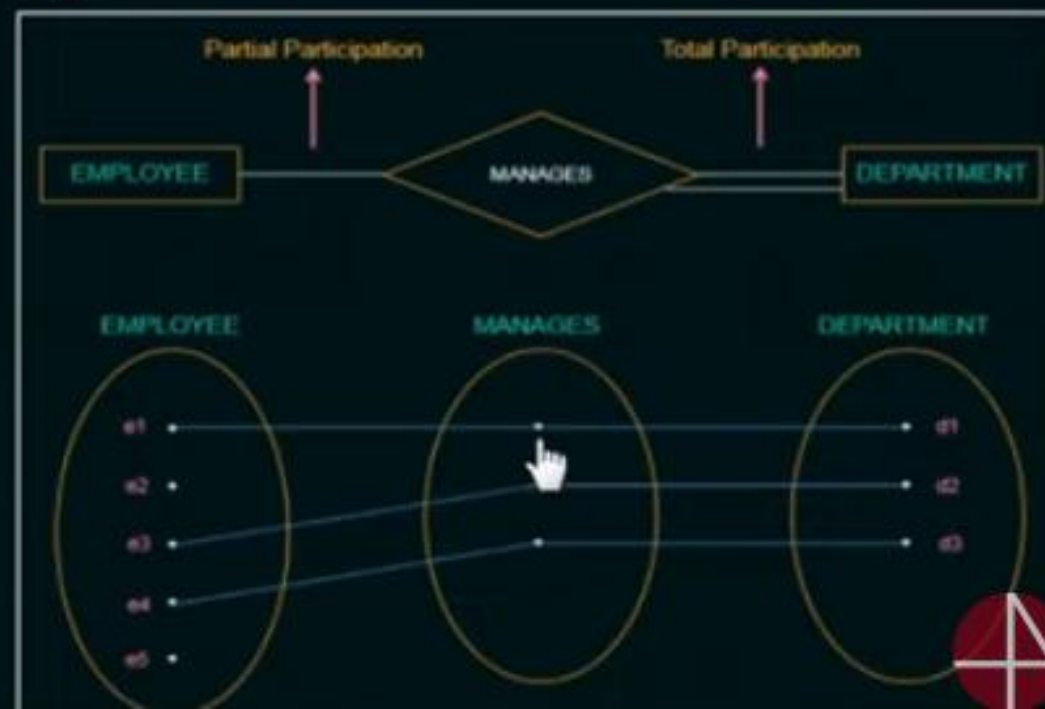


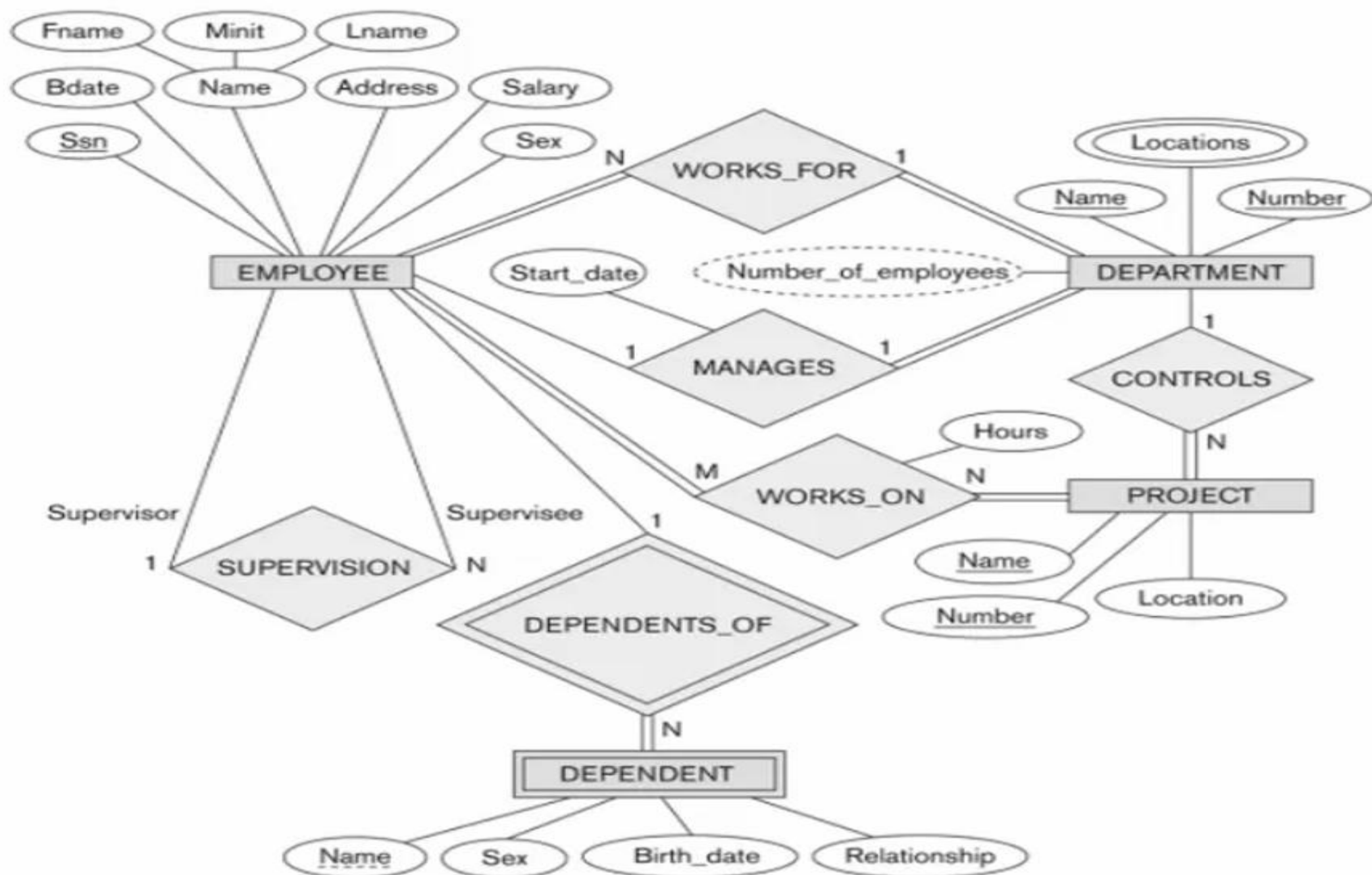
❖ Relationships

★ Relationship Constraints:

2. Participation Constraints

- Specifies whether existence of an entity depends on its being related to another entity.
- 2 types: **Total** participation & **Partial** participation.





ALTERNATIVE (MIN, MAX) NOTATION FOR RELATIONSHIP STRUCTURAL CONSTRAINTS:

- Specifies that each entity e in E participates in at least min and at most max relationship instances in R at any point in time.
- $Min=0$ implies partial participation, whereas $min>0$ implies total participation.
- $min=0, max=n$ (signifying no limit)
- Must have $min \leq max, min \geq 0, max \geq 1$
- Example:
 - A department has exactly one manager and an employee can manage at most one department.
 - Specify (0,1) for participation of EMPLOYEE in MANAGES
 - Specify (1,1) for participation of DEPARTMENT in MANAGES

- An employee can work for exactly one department but a department can have any number of employees.
 - Specify (1,1) for participation of EMPLOYEE in WORKS_FOR
 - Specify (0,n) for participation of DEPARTMENT in WORKS_FOR

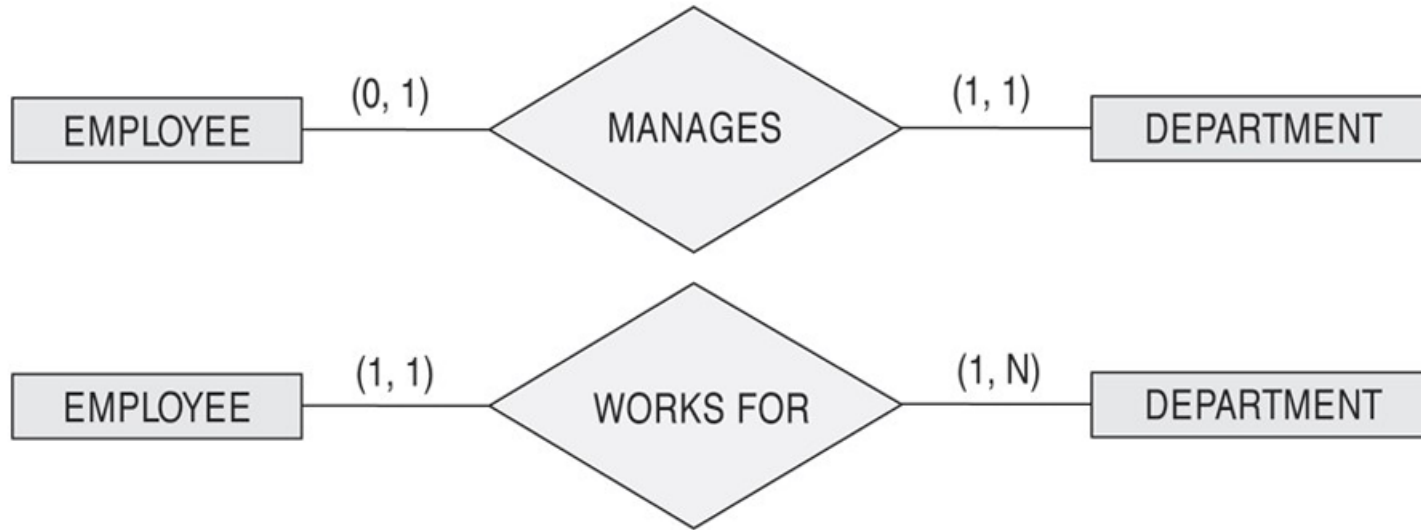


Figure 3.15

ER diagrams for the company schema, with structural constraints specified using (min, max) notation and role names.

